Esercitazioni - Design Pattern: State

Molka Gharbaoui

Descrizione del pattern

Descrizione del pattern

- Permette a un oggetto di cambiare comportamento quando cambia il suo stato interno.
- Offre un modo per implementare un automa a stati mantenendo più aderenza ai principi dell'object-orientation.
- Definisce una classe diversa per ogni stato dell'automa contenente la logica da eseguire in corrispondenza di quello stato.

Descrizione del pattern

- Ogni stato mantiene anche i riferimenti agli stati adiacenti corrispondenti a una transizione.
- Aggiungere un nuovo stato corrisponde a definire una nuova classe con impatti limitati sulle altre.
- Nel corso della vita dell’oggetto, possono essere utilizzate diverse implementazioni senza che l’utente se ne accorga

Descrizione del pattern

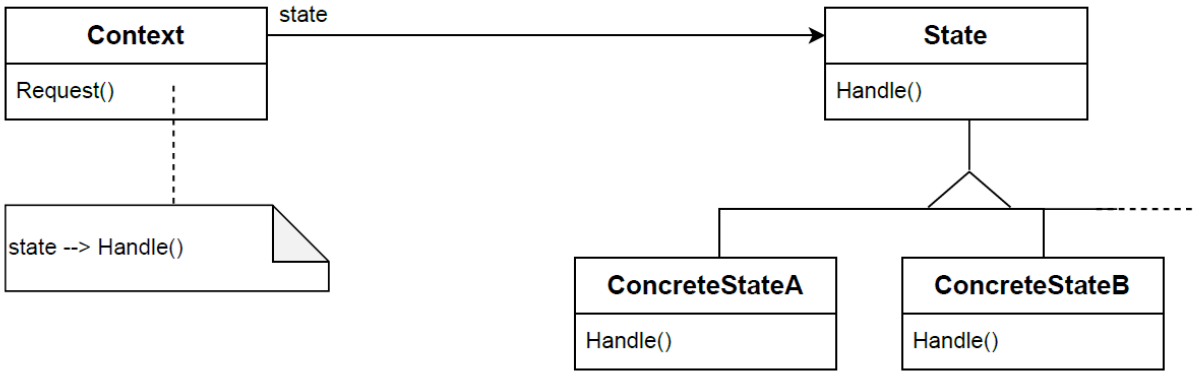
- In Java, una implementazione procedurale di una variazione del comportamento in funzione dello stato prevederebbe una istruzione tipo ***switch***, o una catena di ***if-then-else***.
- Il Pattern State raggruppa le parte delle operazioni sotto le istruzioni condizionali in classi separate, organizzando il codice in classi più snelle, semplici da leggere e da mantenere.

Descrizione del pattern

- I partecipanti al Pattern sono:
  - State è l'interfaccia che definisce le operazioni da eseguire;
  - ConcreteState1,..., ConcreteStateN sono le classi che implementano State e corrispondono ai vari stati dell'automa;
  - Context è la classe che maschera e gestisce i vari stati dell'automa. I clients agiscono su Context e non vedono gli oggetti State.

Struttura del pattern

Struttura del pattern



Struttura del pattern

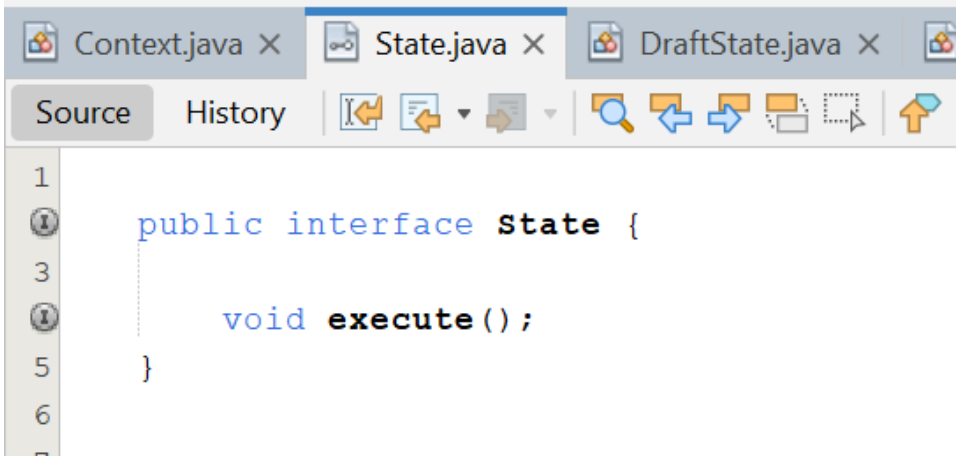
- Vantaggi:
  - Incapsula i comportamenti associati ad uno stato in un oggetto e la logica di transizione tra stati in uno oggetto.
  - Consente un controllo accurato degli stati, evitando stati inconsistenti
  - Semplifica le tabelle di stato/transizione
- Svantaggi:
  - Aumenta il numero degli oggetti rispetto all’uso di uno switch o di una catena if/else

Esempi

Esempi

- Esempio 1: Implementare un programma che consente di gestire dei contenuti considerando 3 stati: draft, published e deleted.

Esempi


Context.java
State.java
DraftState.java
Source
History
1
public interface State {
3
void execute();
5
}
6

Esempi


Context.java
State.java
DraftState.java
DeletedState.java
PublishState.java
Source
History
1
2 public class DraftState implements State {
3
4 @Override
6 public void execute() {
6 System.out.println(x: "Contenuto salvato in stato bozza");
7 }
8
9 }

Esempi

Context.java | State.java | DraftState.java | DeletedState.java | PublishState.java

Source | History

```
1
2   public class PublishState implements State {
3
4       @Override
5       public void execute() {
6           System.out.println( x: "Contenuto salvato in stato pubblicato");
7       }
8
9   }
10
```

Esempi

Context.java | State.java | DraftState.java | **DeletedState.java** | PublishState.java

Source | History

```
1
2   public class DeletedState implements State {
3
4       @Override
5       public void execute() {
6           System.out.println( x: "Contenuto salvato in stato cancellato");
7       }
8
9   }
10
```

Esempi

Context.java | State.java | DraftState.java | DeletedState.java

Source | History

```
1
2   public class Context implements State {
3
4       private State state;
5
6       public State getState() {
7           return this.state;
8       }
9
10      public void setState(State state) {
11          this.state = state;
12      }
13
14      @Override
15      public void execute() {
16          this.state.execute();
17      }
18
19  }
```

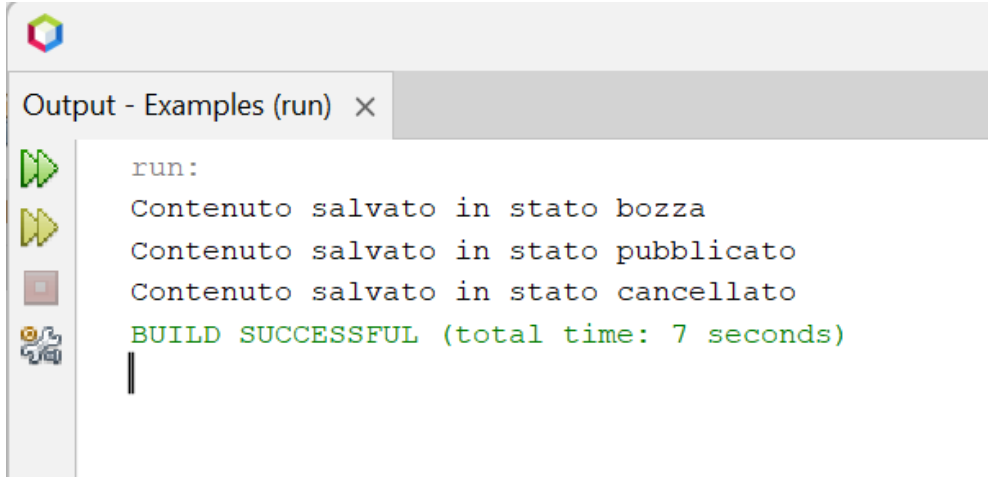
Esempi

Context.java | State.java | DraftState.java | DeletedState.java | PublishState.java

Source | History

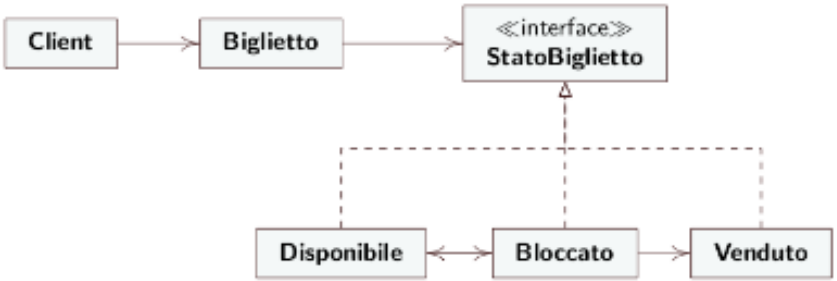
```
1
2   public class Esempio {
3
4       public static void main(String[] args) {
5           Context context = new Context();
6           State darftState = new DraftState();
7           State publishState = new PublishState();
8           State deletedState = new DeletedState();
9
10          context.setState( state:darftState);
11          context.execute();
12
13          context.setState( state:publishState);
14          context.execute();
15
16          context.setState( state:deletedState);
17          context.execute();
18      }
19
20  }
21
```

Esempi


Output - Examples (run)
run:
Contenuto salvato in stato bozza
Contenuto salvato in stato pubblicato
Contenuto salvato in stato cancellato
BUILD SUCCESSFUL (total time: 7 seconds)

Esempi

- Esempio 2: Scrivere il codice java che implementa il design pattern State considerando il caso di un biglietto che può avere 3 stati: Disponibile, Bloccato o Venduto.



Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java

Source | History

```
1
   public interface StatoBiglietto {
3
       public void mostra();
5
       public StatoBiglietto intesta(String s);
7
       public StatoBiglietto paga();
9
       public StatoBiglietto cancella();
11 }
12
```

Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java | Biglietto.java | Client.java

Source | History

```
1
2   public class Disponibile implements StatoBiglietto{
3
4       @Override
5       public void mostra(){  }
6
7       @Override
8       public StatoBiglietto intesta(String s){
9
10          System.out.println( x: "[DISP] Cambia stato da Disponibile a Bloccato");
11          return new Bloccato().intesta(s);
12      }
13
14      @Override
15      public StatoBiglietto paga(){
16
17          System.out.println( x: "[DISP] Non si può pagare, bisogna prima intestarlo");
18          return this;
19      }
20
21      @Override
22      public StatoBiglietto cancella(){
23
24          System.out.println( x: "[DISP] Lo stato era già Disponibile");
25          return this;
26      }}
27
```

Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java | Biglietto.java | Client.java

Source | History

```
1  public class Bloccato implements StatoBiglietto{
2
3      private String nome;
4
5      public void mostra(){
6
7          System.out.println("[BLOC] Nome bloccato: "+nome);
8      }
9
10     public StatoBiglietto intesta(String s){
11
12         System.out.println( x: "[BLOC] Inserito nuovo intestatario");
13         nome = s;
14         return this;
15     }
16
17     public StatoBiglietto paga(){
18
19         System.out.println( x: "[BLOC] Cambia stato da bloccato a venduto");
20         return new Venduto( n: nome).paga();
21     }
22
23     public StatoBiglietto cancella(){
24
25         System.out.println( x: "[BLOC] Cambia stato da bloccato a disponibile");
26         return new Disponibile();
27     }
28 }
```

Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java | Biglietto.java | Client.java

Source | History

```
1  import java.time.LocalDateTime;
2  public class Venduto implements StatoBiglietto{
3      private final String nome;
4      private LocalDateTime dataPagamento;
5      public Venduto(String n){
6          nome = n;}
7      @Override
8      public void mostra(){
9          System.out.println("[VEND] Nome: "+nome);}
10     @Override
11     public StatoBiglietto intesta(String s){
12         System.out.println( x: "[VEND] Non può cambiare nome nello stato venduto");
13         return this;}
14     @Override
15     public StatoBiglietto paga(){
16         if(dataPagamento == null){
17             dataPagamento = LocalDateTime.now();
18             System.out.println( x: "[VEND] Pagamento effettuato");}
19         else
20         System.out.println( x: "[VEND] Il biglietto è già stato pagato");
21     return this;}
22     @Override
23     public StatoBiglietto cancella(){
24         System.out.println( x: "[VEND] Non può cambiare stato da venduto a disponibile");
25         return this;}}
26
```

Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java | Biglietto.java

Source | History

```
1
2   public class Biglietto {
3
4       private String codice = "XYZ";
5       private int prezzo = 100;
6
7       private StatoBiglietto sb = new Disponibile();
8
9       public void mostra(){
10
11          System.out.println("Prezzo: "+prezzo+ " codice: "+codice);
12          sb.mostra();
13      }
14
15      public void prenota(String s){
16          sb = sb.intesta(s);
17      }
18
19      public void cancella(){
20          sb = sb.cancella();
21      }
22
23      public void compra(){
24          sb = sb.paga();
25      }
26  }
27
```

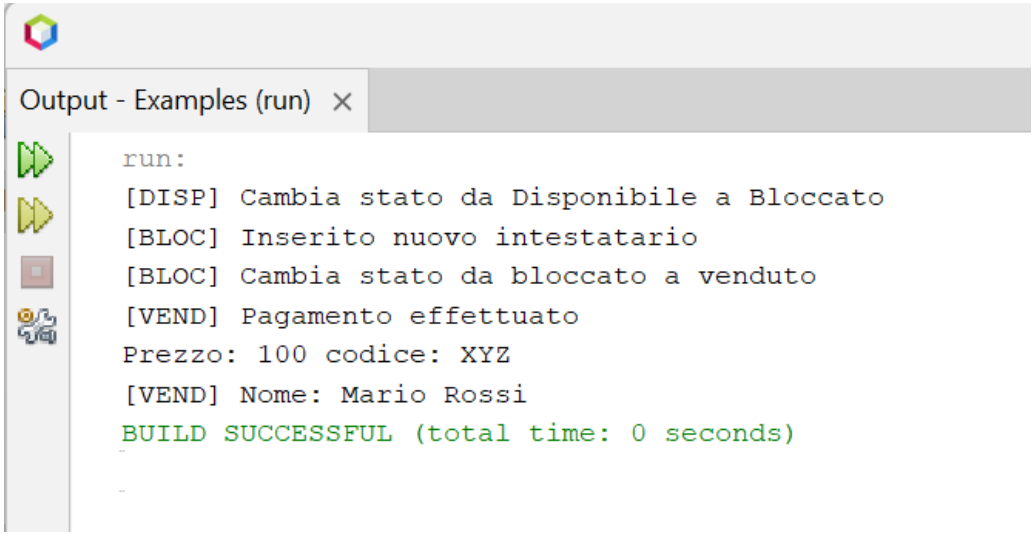
Esempi

StatoBiglietto.java | Disponibile.java | Bloccato.java | Venduto.java | Biglietto.java

Source | History

```
1
2   public class Client {
3
4       private static Biglietto b = new Biglietto();
5
6       public static void main(String[] args){
7
8           usaBiglietto();
9       }
10
11      private static void usaBiglietto(){
12
13              b.prenota( s:"Mario Rossi");
14              b.compra();
15              b.mostra();
16      }
17
18      private static void nonUsaBiglietto(){
19
20              b.compra();
21              b.cancella();
22              b.prenota( s:"Mario Mossi");
23      }
24
25  }
26
```

Esempi


Output - Examples (run)
run:
[DISP] Cambia stato da Disponibile a Bloccato
[BLOC] Inserito nuovo intestatario
[BLOC] Cambia stato da bloccato a venduto
[VEND] Pagamento effettuato
Prezzo: 100 codice: XYZ
[VEND] Nome: Mario Rossi
BUILD SUCCESSFUL (total time: 0 seconds)